

```
$ cat notes.md
```

DevOps & Cloud [SWE] — Class 14 Notes

LECTURE TITLE ConfigMaps, Secrets & Ingress (see the note below — the folder name and the transcript header both say "Helm", and Helm was not taught)

DATE/TIME 17 September 2026, 2:00 PM

DURATION 120 minutes

INSTRUCTOR Nensi Ravaliya

BIG IDEA Ship one image everywhere and change only the config around it — ConfigMap for the harmless values, Secret for the dangerous ones — then put one Ingress in front of everything instead of paying for a load balancer per service.

SOURCES class transcript + her own repo folder for this session,

`Nency-Ravaliya/devops-heros` → `session-12-ingress-configmaps-secrets` (the `01-configmap` ... `04-full-demo` folders she screen-shared, plus her `lab.md` and `troubleshooting/` notes).

WHAT'S INSIDE — 16 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 📅 Housekeeping — A Deliberately Strict Open
- 02 💬 The Kubernetes Recap Drill
- 03 📁 ConfigMap — Why It Exists
- 04 🗑️ Secret — The Same File, One Line Different
- 05 🏢 How Secrets Actually Work in a Real Company
- 06 🧑‍🔬 Lab 1 — ConfigMap & Secret Hands-On
- 07 💰 Ingress — The Akbar & Birbal Story
- 08 🕒 The Ingress File — Rules, and Two Kinds of Routing
- 09 🚦 Ingress vs Ingress Controller
- 10 🧑‍🔬 Lab 2 — The Full Demo, All Three Together
- 11 ⚠️ The Trailing-Newline Bug
- 12 🔄 Updating a ConfigMap — The Pods Don't Care
- 13 📄 Cheat-Sheet
- 14 📝 Homework
- 15 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. theme: Moss

 WATCH OUT

Read this first — the folder name is wrong again.

The folder says `Class-14-Helm` and the transcript header says `Lecture Title: Helm`. **Helm was not taught.** It was named once, in passing, as a place you must *not* put secrets. The class was:

1. a 20-minute Kubernetes recap drill · 2. **ConfigMap** · 3. **Secret** (including how real companies actually do it) · 4. **Ingress** (path-based + host-based routing) · 5. a full hands-on demo tying all three together.

This is the third title mismatch in a row (Class 12 and Class 13 were both wrong too). **Helm is still ahead of you.**

 NOTE

Her session numbering vs. your folder numbering. In her repo this class is `session-12-ingress-configmaps-secrets`. Your folder is `Class-14`. Don't try to reconcile the numbers — go by content. Everything in §6, §10, §11 and §12 below is cross-checked against the real YAML in that folder, not just the audio.



00

⚡ The 60-Second Version

1. **One image, many environments.** Your code must be *identical* in dev, staging and prod. Only the **configuration** changes. That one rule is why ConfigMap and Secret exist.
2. **ConfigMap = non-sensitive, environment-specific data.** `LOG_LEVEL` , `ENVIRONMENT` , `PORT` , currency, timeouts, feature flags.
3. **Secret = sensitive, environment-specific data.** DB passwords, JWT tokens, API keys — *"whatever you put in your `.env` file."*
4. **Both files are the same file.** `apiVersion` · `kind` · `metadata` · `data` — the usual `spec` block is replaced by `data` . Secret adds exactly **one** extra line: `type: Opaque` .
5. **Base64 is encoding, not encryption.** `base64 --decode` reverses it in one command. Secrets are protected by **RBAC**, not by the encoding.
6. **Nobody puts real passwords in `secret.yaml` .** Production chains a variable out to **AWS Secrets Manager**, **Azure Key Vault** or **HashiCorp Vault** via the CI/CD pipeline.
7. **Ingress exists to save money.** Five services = five cloud load balancers = five monthly bills. One Ingress = **one** bill, and it fans traffic out to all five.
8. **Ingress is only a rulebook.** The **Ingress Controller** (e.g. NGINX) is the thing that actually enforces the rules. That distinction is this class's homework — and §9 answers it.
9. **Two routings: path-based** (`yatri.local/` vs `yatri.local/api`) and **host-based** (`portal.campus.local` vs `api.campus.local`). You can combine them.



01

📅 Housekeeping — A Deliberately Strict Open

She opened hard, and it's worth recording because the reason was stated plainly.

The homework situation. Every Kubernetes task so far was due **end of day**, in the `README.md` of your own repo, at the same link she'd already shared (updated with the Kubernetes lecture names). Today's task is due **tomorrow**.

The immediate ask — before anything was taught:

🗣️ SAID IN CLASS

"I need all the output for 5 services. NodePort, ClusterIP, LoadBalancer, ExternalName and Headless. And last command you can just type `kubectl get svc ...` put that screenshot in README.md file and then add your repo in chat."

Deadline for that: **2:35–2:40**, live, in the chat. She then went round the room asking people by name to ping their repo (*"Aditya is our first target"*) because too few were coming in.

The classroom rules, stated once:

Rule	Consequence
Phone use in class	Go outside
Talking in class	Go outside
Once you go out	"No need to enter into the class again"
Attendance	Taken mid-class , not at the end

SAID IN CLASS

The reason she gave, not a mood: *"I'm not getting the responses from your side regarding assessment, regarding homework, and then I get some feedbacks that 'I don't understand'." Then, in Hindi: "To understand deeply, you first need to know the prerequisites. If you don't even know what a pod is, you can't understand a deployment."*

The lab that wasn't ready. She had handed the program team a zip to load into the cloud lab environment; it wasn't done in time, so **there was no lab instance for this class.**

CLASS MOMENT

Class moment. She'd also tried to demo the Azure DevOps portal live and couldn't log into her own Azure account — *"I tried it in Section B morning class, I'm not able to log in."* So the Azure Key Vault walkthrough in §5 was done from memory and screenshots rather than the real portal. Promise made: server access **from next session**, plus a lab covering exactly this demo.



02



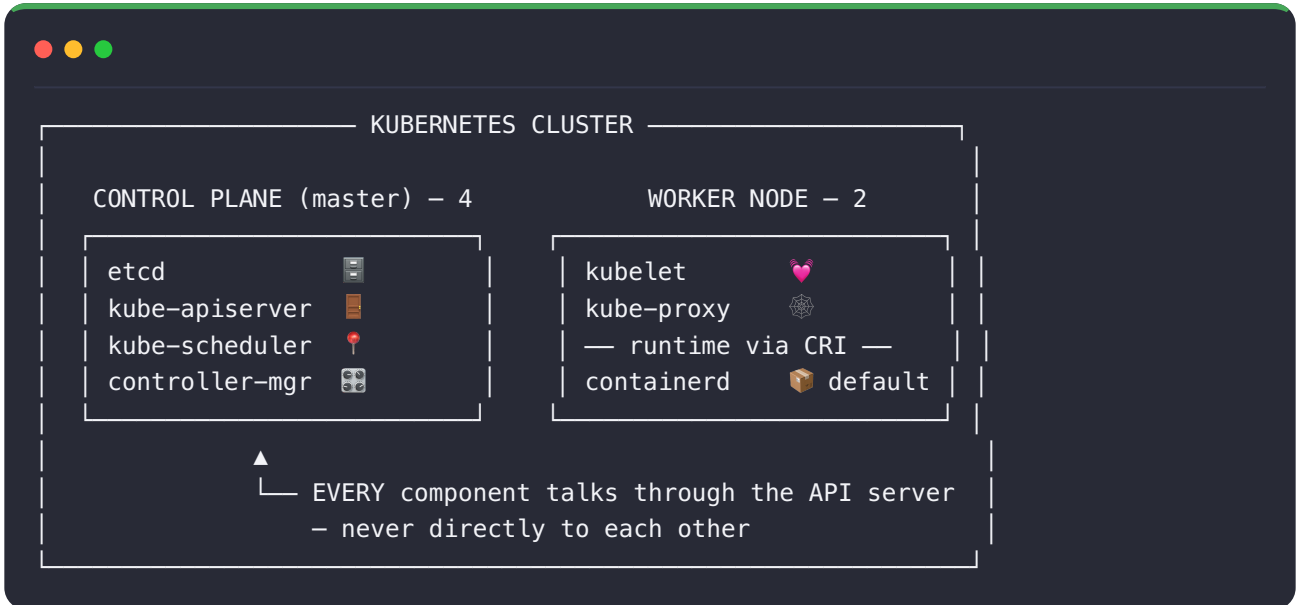
The Kubernetes Recap Drill

Twenty minutes of rapid-fire questions before any new material. Same reasoning as last class: she does not want Kubernetes to end and everyone to have to relearn it.

NOTE

This is the fourth pass over the architecture (Class 9 introduced it, Class 10 walked the `kubectl apply` journey, Class 13 drilled it as Q&A, this one drills it again and then extends into services and workloads). **These notes re-teach it in full every time rather than pointing backwards** — that repetition is the whole point, and the fourth telling is often the one that sticks.

2.1 The component drill



Question	Answer given	✓
How many components in the master node?	4 — etcd, API server, controller, scheduler	✓
How many in a worker node?	2 — kube-proxy and kubelet	✓
Default container runtime interface?	containerd	✓
Job of etcd?	"Key-value pair" database	✓
Job of the scheduler?	"To schedule a pod in a particular node"	✓
Job of the API server?	Handler — plus the point she pushed for: it is the only point of contact	✓
Job of the controller?	Manage the workload on the worker nodes	✓
Name a controller	ReplicaSet controller, node controller	✓
Is there a "deployment controller"?	Class guessed yes → she told them to go search it	?
Is there a "StatefulSet controller"?	"I haven't heard of that — go search"	?
Cloud controller?	"That's a separate component altogether"	✓

etcd — what actually lives in it:

SAID IN CLASS

"It's the database that is storing every object data. Let's suppose pod is there, node is there, deployment, ReplicaSet, DaemonSet, StatefulSet — anything related to our Kubernetes cluster, the data will store in etcd."

The API server — the second half of the answer she wanted:

✓ LOCK THIS IN

The API server is the only point of contact for every component inside the cluster. Components cannot talk to each other. They communicate through the API server only.

The controller — plural, not singular:

🗣️ SAID IN CLASS

"Different different types of controllers are there — ReplicaSet controller, node controller, ingress controller, job controller. All the controllers are managing my worker nodes' workload. To manage the workload we need a controller. That's it."

💡 GOOD TO KNOW

On the two she wasn't sure about. She was right to send you to check rather than guess. For the record: Kubernetes ships a **Deployment controller** and a **StatefulSet controller** — they live inside `kube-controller-manager` alongside the ReplicaSet and node controllers. The **ingress controller** she named is the odd one out: it is *not* part of `kube-controller-manager`, it's a separate pod you install yourself. That difference is exactly this class's homework (§9).

kubelet — the heartbeat:

🗣️ SAID IN CLASS

"Immediately, without even thinking, your subconscious mind will tell you — kubelet sends the heartbeat of your pod to the API server. Then kubelet is the one who is managing your container, who is creating your container, who is deleting your container, with the command of the controller."

📌 NOTE

The ASR wrote "*kubect!*" throughout that answer. It is **kubelet** — the node agent. `kubect!` is the CLI on your laptop. Same first six letters, completely different things.

| 2.2 The `kubect!` get family

Every object, same verb:

```
kubectl get pods
kubectl get nodes
kubectl get deploy      # short for deployment
kubectl get rs          # short for replicaset
kubectl get daemonset
kubectl get statefulset
kubectl get configmap    # ← new today
kubectl get secrets      # ← new today
```

And the creation side is equally uniform, which is the point she was building to:

✓ LOCK THIS IN

One command creates all of them. *"Whether it's a ReplicaSet, whether it's a Deployment, whether it's a ConfigMap, Secret, Ingress — anything. If it's a YAML file, then `kubectl apply -f <file>`. There is the only way to apply a YAML file."*

So today's two new objects are **two new YAML files**, not two new workflows.

2.3 The five Service types

Question	Answer
How many Service types?	5
Name them	ClusterIP (default), NodePort , LoadBalancer , ExternalName , Headless
Difference between ClusterIP and Headless in the YAML?	Headless sets <code>clusterIP: None</code> . ClusterIP sets <code>type: ClusterIP</code> . <i>"This is the only difference."</i>
Difference between ClusterIP and NodePort?	ClusterIP is internal only — not reachable from a browser outside the cluster. NodePort needs <code>type: NodePort</code> and a <code>nodePort</code> : number (30080, 30081...), then it's reachable on the Minikube/server IP or <code>localhost:<nodePort></code> .
Which type shows an EXTERNAL-IP ?	ExternalName — check with <code>kubectl get svc -o wide</code> , the external IP column sits after CLUSTER-IP
What does LoadBalancer show locally?	<code><pending></code> — <i>"Because in local, maybe it will not work, then you will see the pending."</i>

😊 CLASS MOMENT

Callback to last class. Their ExternalName demo pointed at `api.github.com`, and she then repointed it at her own domain mid-demo. Her portfolio lives at `nancy.yatricloud.com` — a subdomain she comes back to in §8 when explaining host-based routing.

2.4 The workload drill

Question	Answer
StatefulSet vs DaemonSet?	StatefulSet runs stateful applications. DaemonSet puts one pod on every node .
StatefulSet vs Deployment?	"Most frequently asked question in interview."
How many deployment strategies?	4 — starting with rolling update

The StatefulSet vs Deployment answer, in full:

✓ LOCK THIS IN

Deployment → **stateless**. **StatefulSet** → **stateful**.

"Frontend and backend — to deploy those services you will use **Deployment** as the **kind**, because those are **stateless** applications. You don't need to make sure that after deleting this pod, I need the same pod with the storage, with all my data. No. That's why we call it **stateless**."

"But for a **stateful** application — let's suppose a **DB**. **DB** we need to store the data, we need to take the storage. Even if my container goes down or terminates, it will create the same name of container with the help of **StatefulSet**. **mysql-0**, **mysql-1**, **mysql-2** — three pods were created, then I deleted **mysql-0** and again the same pod will be created. We did this hands-on last time."

So: **MySQL, Postgres, databases** → **StatefulSet**. Everything else → **Deployment**.



03 📁 ConfigMap — Why It Exists

She refused to open the YAML until the class could say *why*.

3.1 The three-environment problem

You have **dev**, **stage** and **production**. You have one environment variable — the same one you already met in a Dockerfile as `ENV ENVIRONMENT=production`.

deployment.yaml (dev)

```
... 60 identical lines
ENVIRONMENT = dev
LOG_LEVEL   = DEBUG
... 20 identical lines
```

deployment.yaml (stage)

```
... 60 identical lines
ENVIRONMENT = stage
LOG_LEVEL   = INFO
... 20 identical lines
```

deployment.yaml (prod)

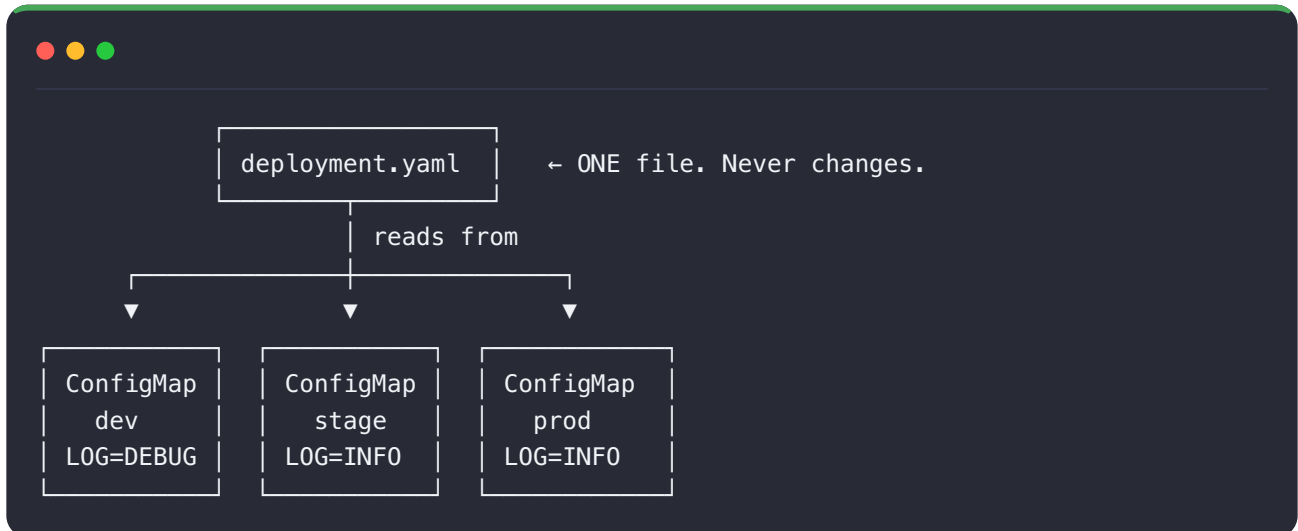
```
... 60 identical lines
ENVIRONMENT = production
LOG_LEVEL   = INFO
... 20 identical lines
```

↑ three near-identical files, drifting apart, forever

SAID IN CLASS

"Application we have the same, and then the code also we have the same — only few variables will change. It's one type of problem that we are facing in Kubernetes."

The fix: lift those variables out into a separate object.



LOCK THIS IN

The whole reason, in her words: "Environment-specific data you need to separate from your code base. In your code you don't want to use like `username = localhost`."

PICTURE THIS

Picture this — the hotel key card.

The room is the same room for every guest. You don't rebuild the room when a new guest arrives; you issue a **different key card**. The image is the room. The ConfigMap is the key card.

3.2 The LOG_LEVEL tangent

She stopped to poll the room on `LOG_LEVEL`, because it's the cleanest example of a variable that *must* differ per environment:

Environment	LOG_LEVEL	Why
dev	DEBUG	you want every message, verbose output, while you're building
stage	INFO	quieter, closer to real
production	INFO	nobody wants debug noise in prod logs

💡 GOOD TO KNOW

If you've done backend development you already know this variable. It's the standard dial that decides how chatty your application's logs are: `DEBUG` → `INFO` → `WARN` → `ERROR`. She flagged it as "a default variable — if you know development, then you know this variable."

3.3 Writing the file — the one structural change

Every Kubernetes YAML you've written so far has **four** top-level fields:

```
apiVersion:  ← which API group/version
kind:        ← what object this is
metadata:    ← name, labels, namespace
spec:        ← the desired state
```

✅ LOCK THIS IN

For ConfigMap and Secret, you replace `spec` with `data`. That's the entire structural difference.

```
apiVersion:
kind:
metadata:
data:        ← instead of spec
  KEY: "value"
  KEY: "value"
```

🗣️ "We just replace `spec` with `data`, that's it. Smallest file ever you see in Kubernetes. Very easy concept."

Her actual file (`04-full-demo/configmap.yaml` from her repo):

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: yatri-app-config
  namespace: default
  labels:
    app: yatri-app
data:
  ENVIRONMENT: "production"
  LOG_LEVEL: "INFO"
  APP_PORT: "5000"
  DEFAULT_CURRENCY: "INR"
  MAX_BOOKING_DAYS: "30"

```

NOTE

Sourcing note. The `01-configmap/app-config.yaml` version in her repo is identical except the third key is named `PORT` rather than `APP_PORT`. The demo in §10 uses `APP_PORT`. Harmless, but if you copy from the wrong folder your `describe` output won't match hers.

3.4 What goes in — and what never does

✓ Belongs in a ConfigMap

`LOG_LEVEL`, `ENVIRONMENT`

Port numbers, API base URLs

Feature flags (`FEATURE_DARK_MODE: "true"`)

Timeouts, currency, limits

A whole `nginx.conf`, mounted as a file

✗ Never in a ConfigMap

Database passwords

JWT tokens

API keys / OAuth secrets

TLS private keys

Anything from your `.env`

⚠ WATCH OUT

Three facts about ConfigMaps that bite people (from her repo's `01-configmap/README.md`):

- **Updating a ConfigMap does not restart your pods.** The new value sits there unused until the pod restarts. §12 demonstrates this.
- **Size limit is 1 MiB.** It is not a file store.
- **A ConfigMap referenced by `envFrom` that doesn't exist** will stop your pod from starting — it lands in `CreateContainerConfigError`.

INTERVIEW ALERT

Interview alert — the three things to be able to say about ConfigMap: *"why you need to use it, where you need to use it, and which type of variables you can put inside it."* She listed those three explicitly as the examinable set.



04



Secret — The Same File, One Line Different

4.1 What counts as "sensitive"

She asked the room to define it and got the right answer — *"that cannot be exposed"* — then pushed for concrete examples:

- Database password
- JWT token
- Auth / API keys
- Everything in your `.env` file

LOCK THIS IN

"If you know any kind of application, backend or frontend, there is a `.env` file. Whatever we are putting inside `.env` file — the most sensitive data — those data we store inside `secret.yaml` in Kubernetes. Just naming convention. In YAML format."

4.2 The base64 detour

Before the file, a live terminal demo:

```
echo -n "yatri_admin" | base64
# eWF0cm1fYWRTaW4=
```

CLASS MOMENT

Nobody in the room had met base64. She asked *"Base 8, base 16, base 36, base 64, base 128 — do you study this? Which subject?"* and got blank looks. Then, deciding it was worth the time anyway, she told the story:

Her college professor **Sahista** ran an exercise where the class had to **email her in encoded form**, and she would decode each one. *"She was telling us: you mail me in your encoded language till I'm able to decrypt you guys."* Her point for this class: **you only remember a concept once you've used it for something.**

Then the reversal:

```
echo -n "eWF0cmJfYWRTaW4=" | base64 --decode
# yatri_admin
```

😊 CLASS MOMENT

The live blunder, and it taught the exact right lesson. She first tried *"brute force"* — re-ran the same command with the encoded value and no `--decode` flag — and got **another layer of base64**, because encoding an encoded string just encodes it again. She caught it out loud: *"See, but do you know what happened here? I just put my output inside the same thing — again encrypting in base64."* Then added `--decode` and it came back clean.

🔍 CLARIFIED

Clarified, since she said it twice. She described base64 as *"encrypting your data in 64 bits."* Two corrections, and both matter:

She said	Actually
"encrypting"	Encoding. Encryption needs a key and is <i>meant</i> to be unreadable without it. Base64 needs nothing — <code>base64 --decode</code> is the whole attack.
"64 bits"	64 characters. It's the size of the alphabet (<code>A-Z</code> , <code>a-z</code> , <code>0-9</code> , <code>+</code> , <code>/</code>), not a bit length.

She reaches the right conclusion two minutes later on her own (*"can we use the password like this in our file? No"*), so **in her assessment answer both facts, in this order: base64 is encoding, not encryption; Secrets are protected by RBAC, not by the encoding.**

4.3 The file

```
apiVersion: v1
kind: Secret                                # ← change 1: the kind
metadata:
  name: yatri-db-secret
  namespace: default
  labels:
    app: yatri-app
type: Opaque                                # ← change 2: the ONE extra line
data:
  # echo -n "yatri_admin" | base64
  POSTGRES_USER: eWF0cmJfYWRTaW4=
  # echo -n "secretpassword" | base64
  POSTGRES_PASSWORD: c2VjcWV0cGFzc3dvcmQ=
  # echo -n "yatri_production_db" | base64
  POSTGRES_DB: eWF0cmJfcHJvZHVjdGlub19kYg==
```

✓ LOCK THIS IN

ConfigMap vs Secret — the complete difference.

	ConfigMap	Secret
apiVersion	v1	v1
kind	ConfigMap	Secret
metadata	same	same
type:	—	Opaque ← the only extra line
values	plain text	base64-encoded
holds	non-sensitive config	sensitive config

🗣️ "Both the files are same. Both the files are same. Just **kind** will change, and one line will be added in your **secret.yaml** — **type**."

💡 GOOD TO KNOW

Opaque just means "arbitrary key-value data" — it's the default and the one you'll use for 95% of Secrets. Kubernetes has typed variants for special cases; you'll meet kubernetes.io/tls in §8.3 when an Ingress terminates HTTPS.

SAID IN CLASS

Reality check she gave immediately after showing the file: *"Can we use the password like this in our file? No. It's a manual way, we are not using it."* Which leads directly to §5.



05 How Secrets Actually Work in a Real Company

This was flagged as *"intermediate to advanced"* and it's the most interview-relevant block in the class.

5.1 The four ways, ranked

#	Method	Verdict
1	AWS Secrets Manager	✓ standard on AWS
2	Azure Key Vault	✓ standard on Azure
3	HashiCorp Vault	✓ <i>"the most widely used"</i> , cloud-agnostic
4	Typing base64 into <code>secret.yaml</code>	✗ <i>"we are never, never ever using"</i>

SAID IN CLASS

Her own workplace, as a concrete data point: *"In our organization we are using Azure Key Vault for dev environment, and for production environment we are using HashiCorp Vault. HashiCorp Vault has some few extra features — when we are going to learn Terraform, I'll tell you."*

How all three work, identically in shape:

1. Create the vault/manager service in the cloud portal.
2. Put in a **secret name** and a **secret value**, then lock it.
3. Give your application (or pipeline) **credentials/permissions** to read from that service.
4. At deploy time the pipeline **fetches** the secret and injects it — *"without sacrificing your password security."*

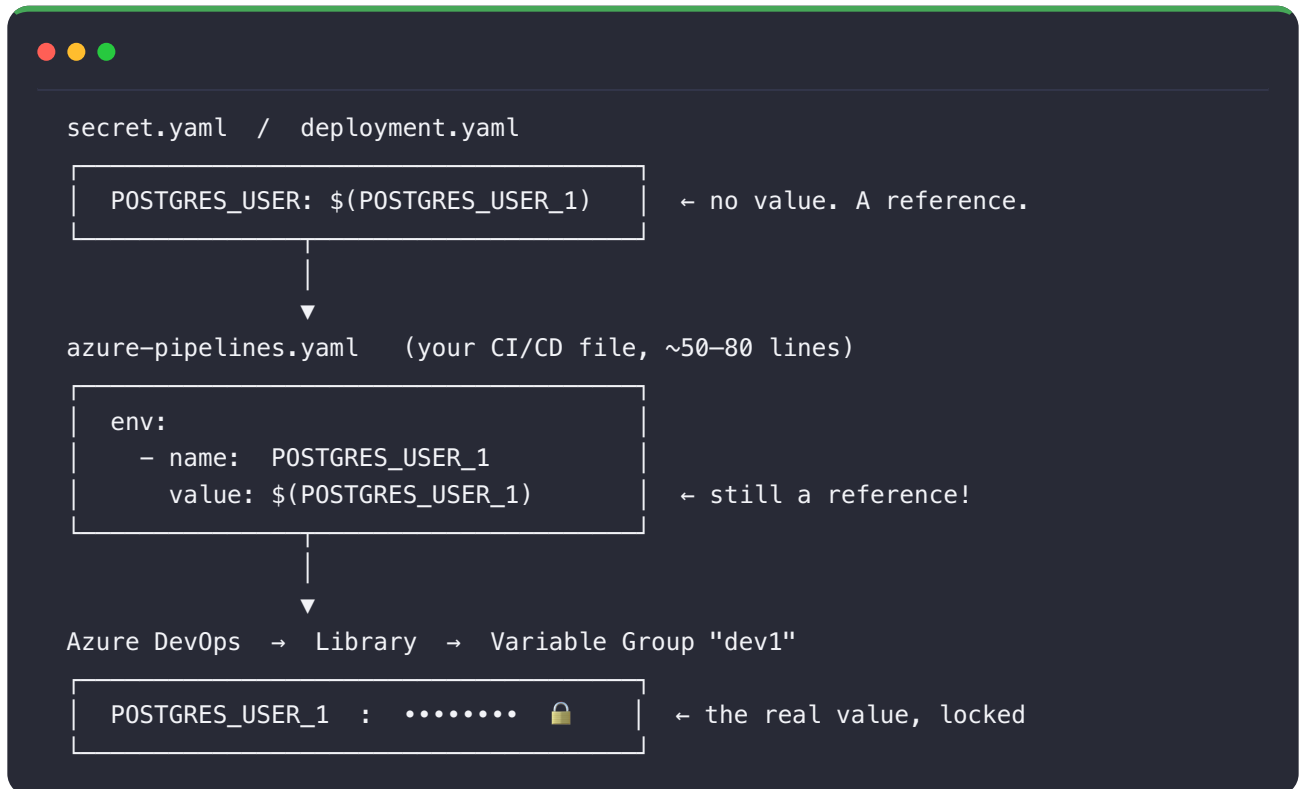
WATCH OUT

Three places a secret must never live, stated together: - ✗ in your **code** - ✗ in your `secret.yaml` committed to git - ✗ in your **Helm chart** — *"You can't add all the secrets in Helm chart."*

📌 That Helm mention is the **only** time Helm appears in this class, which is exactly how much of "Helm" the `Class-14-Helm` folder actually contains.

5.2 The variable chain — the standard method

This is the part worth memorising. A secret value is never written anywhere: it is a **variable pointing at a variable pointing at a vault**.



SAID IN CLASS

"One variable is redirecting to another variable... I did not expose my value. Can you see, guys — I exposed my password? No."

WHY IT WORKS

Why the chain has three links and not two. Each link is owned by a different person: the **developer** writes the YAML, the **pipeline** is owned by the platform team, and the **vault** is owned by whoever holds production access. A developer can read and edit every file in the repo and still never learn the password. That separation is the entire point — not the encoding.

5.3 The Azure DevOps Library, step by step

She walked the portal path from memory (the live login failed):

1. portal.azure.com → search **Key Vault** → create a vault. (*That's the storage.*)
2. azure.microsoft.com → **Azure DevOps portal** → sign up.
3. Inside it: **Library** — "we call it Azure ADO Library; because it's new for you, you can just spell it Azure Library."
4. Inside Library: **variable groups**. A group is "nothing else, one folder you created" — e.g. `dev1`. It can hold thousands of variables.

- Each variable is **two fields**: the **secret name** (e.g. `POSTGRES_USER`) and the **value** (e.g. her example password `nancy@123`).
- Click the  **lock button**.

⚠️ WATCH OUT

What locking actually does — she laboured this, correctly.

Once locked, **you can never see that value again. Ever.** Not by unlocking, not as admin, not as the person who typed it.

- You **can** overwrite it with a new value.
- After overwriting, it's locked again — still invisible.
- Unlock it and the field reads **empty**, not the password.
- "Password is null — but password is there, password is working."

🎤 "Let's suppose I am the admin. I created that `dev1` group, I created that `POSTGRES_USER`, I updated the password, I locked it. I'm not able to see the password again. I know the password that I know — but I'm not able to see it. That type of security they have."

🎯 INTERVIEW ALERT

Interview alert. "Whenever you are working with `secret.yaml`, you need to know which type of implementation you need to do." The wrong answer is "I base64-encode it and put it in the file." The right answer is the chain in §5.2, named with a real vault.



06



Lab 1 — ConfigMap & Secret Hands-On

Five commands, in her order: **create** → **get** → **describe** → **delete**.



DO THIS YOURSELF

Do this yourself — the ConfigMap half.

Step 1. Apply it.



```
kubectl apply -f configmap.yaml
```



```
configmap/yatri-app-config created
```

Step 2. List it.



```
kubectl get configmap yatri-app-config
```



NAME	DATA	AGE
yatri-app-config	5	8s

Step 3. Read the keys.



```
kubectl describe configmap yatri-app-config
```



```
Name:          yatri-app-config
Namespace:     default
Labels:        app=yatri-app
Data
====
APP_PORT:      5000
DEFAULT_CURRENCY: INR
ENVIRONMENT:    production
LOG_LEVEL:      INFO
MAX_BOOKING_DAYS: 30
```

Step 4. Pull out **one** value — the trick she promised and then ran out of time to show:



```
kubectl get configmap yatri-app-config -o jsonpath='{.data.ENVIRONMENT}'
```



```
production
```

Step 5. Delete it.



```
kubectl delete configmap yatri-app-config
```



DO THIS YOURSELF

Do this yourself — the Secret half.



```
kubectl apply -f secret.yaml
kubectl get secret yatri-db-secret
```



NAME	TYPE	DATA	AGE
yatri-db-secret	Opaque	3	5s

Now `describe` — and notice what it **doesn't** print:



```
kubectl describe secret yatri-db-secret
```



```
Name:          yatri-db-secret
Type:          Opaque
Data
====
POSTGRES_DB:   19 bytes
POSTGRES_PASSWORD: 14 bytes
POSTGRES_USER: 11 bytes
```

The values are masked — only byte counts. Nobody reading over your shoulder gets the password from `describe`.

Now prove base64 isn't security:



```
kubectl get secret yatri-db-secret \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 --decode
```



```
secretpassword
```

6.1 The **DATA** column — her favourite question of the day

She stopped on it and would not move on until someone got it.

NAME	DATA	AGE	NAME	TYPE	DATA	AGE
yatri-app-config	5	8s	yatri-db-secret	Opaque	3	5s
	▲				▲	
5 key-value pairs in the file			3 key-value pairs in the file			

✓ LOCK THIS IN

DATA = how many keys are inside. Five variables in the ConfigMap → **DATA 5** . Three secret name/value pairs → **DATA 3** . Nothing cleverer than that.

😊 CLASS MOMENT

The honesty poll. After explaining the file twice she asked how many people understood. Answer: "5 people." One student said "I don't understand anything." So she went back and did the four-fields walkthrough again — **apiVersion** , **kind** , **metadata** , then **data** instead of **spec** — which is why that explanation appears twice in §3.3 and here. Asking got the re-explanation; not asking wouldn't have.

6.2 Two objects in one command

```
kubectl get configmap,secrets
```

⚠ WATCH OUT

No space after the comma. "I will not add space — no, it will not run. You can try." `kubectl get configmap, secrets` fails; `kubectl get configmap,secrets` works. The same comma syntax works for any mix: `kubectl get deploy,rs,pods` .

😊 CLASS MOMENT

A live error nobody minded. Her first **apply** came back **invalid** — she'd added a line and not saved/updated the file before applying. Diagnosed out loud and re-applied. "What is the problem? I don't know. It's invalid. Why is it invalid?" — then found it. Worth watching how fast a YAML edit-then-forget bites.

6.3 Why `kubectl get all` hides them

Someone ran `kubectl get all` and their ConfigMap and Secret weren't in it.

SAID IN CLASS

Her explanation: "Normally when we are typing the `kubectl get all` command, it will give you only [objects with] `spec` in the file. Where is the `spec` here? In the file there is `data`. So it can't come."

CLARIFIED

Clarified, because the rule she gave will mislead you later. The `spec` -vs- `data` story is a neat mnemonic but it isn't the mechanism. `kubectl get all` does not mean "all resources." It expands to a fixed, hardcoded shortlist — roughly **pods, services, deployments, replicaset, statefulsets, daemonsets, jobs and cronjobs**. ConfigMaps and Secrets aren't on the list; neither are **Ingresses, PVCs, ServiceAccounts or Nodes**, and several of those *do* have a `spec`.

The practical takeaway is the same as hers — don't trust `get all` to show you everything — but the reason is "it's a curated list", not "it only shows things with a spec". Ask for them by name:

```
kubectl get configmap,secret,ingress,pvc
```

For her assessment, the expected answer is simply: `get all` does not include ConfigMaps and Secrets; you must query them explicitly.



07 💰 Ingress — The Akbar & Birbal Story

She told the whole thing as a folk tale, and it works, so here it is intact.

🔥 HARD MODE

The story.

There was a jungle, and a king. The king had **three services** — a frontend, a backend and a DB. Each one had its own load balancer, and each load balancer cost **\$50 a month**.



```
Frontend → Load Balancer 1  $50/mo
Backend  → Load Balancer 2  $50/mo
DB       → Load Balancer 3  $50/mo
```

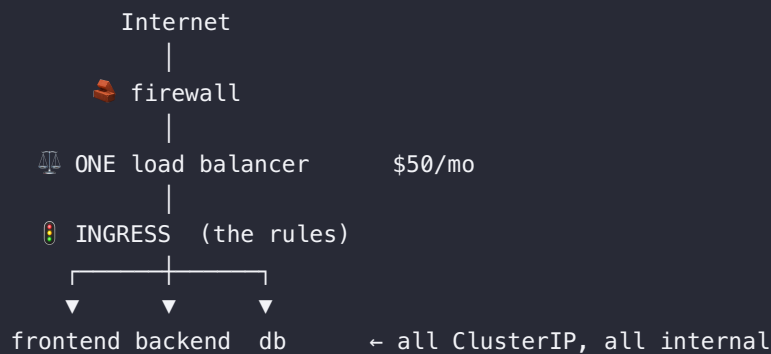
\$150/mo just to let traffic in

The king looked at the bill and asked Birbal: *"Why am I paying for three? Can't we pay just \$50 for one, and use it for all three?"*

😏 **Her aside, mid-story:** *"Ambani will hire the CEO, give you 5 lakhs and tell you that after 6 months I need 10 lakhs. That's it. I don't care — black, white, blue, whatever. I need my 10 lakhs after 6 months."* — i.e. the king states the number and the engineer finds the way. Which is roughly the job.

Birbal found the person in the kingdom who actually knew the answer ("Birbal doesn't know how to code — the one he grabbed knows how to code. **That's basically us, the engineers, at the lower level.**") and came back with it:

Remove all three load balancers. Keep the three services. Put ONE thing on top that receives every request and redirects it to the right service.



"And Raja ho gaye khush." The king was happy, Birbal got his reward, and went home to tell his mother he'd got a raise.

✅ LOCK THIS IN

The one-line takeaway she wanted: *"Why do we need to use Ingress? To save the cost. You can redirect your user request to multiple services from one endpoint. That we call Ingress."*

NOTE

Deliberate repeat. Ingress first came up in **Class 11 §4**, also as a cost argument, but it was never demoed. This class is the full version: the story, the file, both routing types and a working demo. Everything you need is here — you don't need to go back.

The cost argument at five services (from her repo's `03-ingress/README.md`):

Without Ingress	With Ingress
Frontend → LB 1 — \$25/mo	
Backend API → LB 2 — \$25/mo	1 load balancer — \$25/mo
Auth → LB 3 — \$25/mo	↓
Payment → LB 4 — \$25/mo	NGINX Ingress Controller
Admin → LB 5 — \$25/mo	↓ routes to all 5 ClusterIP services
\$125/month	\$25/month

GOOD TO KNOW

Cost isn't the only win, it's just the one that persuades a king. Her README lists three more: users stop seeing ugly URLs like `http://3.15.22.100:30080`; you get **one place to terminate TLS/HTTPS** instead of a certificate per service; and you can route by hostname (`api.myapp.com` vs `myapp.com`), which NodePort simply cannot do.



08 The Ingress File — Rules, and Two Kinds of Routing

SAID IN CLASS

Her framing, and a good revision checklist: *"Why do we need Ingress, what do you mean by Ingress, where do we need to use Ingress, and how can we implement Ingress."* Four questions. §7 answered the first three.

Ingress is just another Kubernetes object — same four fields as everything else:

```

apiVersion: networking.k8s.io/v1  # ← note: NOT v1, NOT apps/v1
kind: Ingress
metadata:
  name: yatri-ingress
spec:
  ingressClassName: nginx          # ← mandatory
  rules:                           # ← everything interesting lives here
    - host: ...

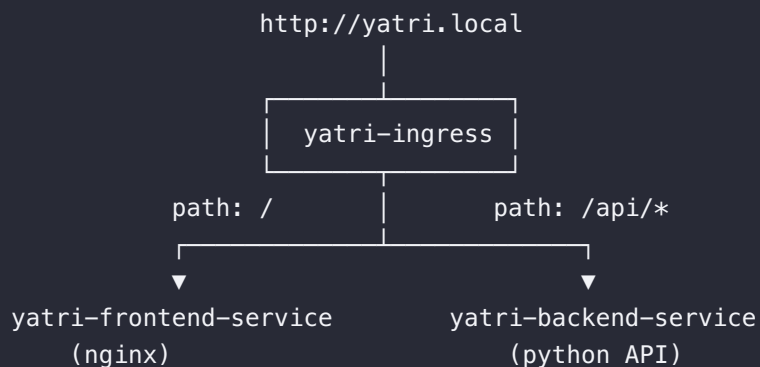
```

⚠ WATCH OUT

ingressClassName is not optional. *"It's necessary to give a class name."* Her repo's interview notes sharpen why: on Kubernetes v1.18+ this field tells the cluster **which controller** should pick up this rule. **Forget it and your Ingress is silently ignored** — no error, no routing, nothing. Just a resource that does nothing.

8.1 Path-based routing

One host. The **path** decides where you land.



Her actual `04-full-demo/ingress.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: yatri-ingress
  namespace: default
  labels:
    app: yatri-app
  annotations:
    # Do not force HTTP -> HTTPS redirect (no TLS cert in this demo)
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
    # Allow regex capture groups in path rules
    nginx.ingress.kubernetes.io/use-regex: "true"
    # Strip /api prefix before forwarding to backend pods
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - host: yatri.local
      http:
        paths:
          - path: /api(/|$)(.*)          # → backend
            pathType: ImplementationSpecific
            backend:
              service:
                name: yatri-backend-service
                port:
                  number: 80
          - path: /                      # → frontend
            pathType: Prefix
            backend:
              service:
                name: yatri-frontend-service
                port:
                  number: 80
```

✓ LOCK THIS IN

The shape to memorise — she said *"anyhow, you need to remember this thing"*:

```
rules:
  - host: <domain>
    http:
      paths:
        - path: <the URL path>
          pathType: <Prefix | Exact | ImplementationSpecific>
          backend:
            service:
              name: <service name>
              port:
                number: <service port>
```

Adding a third path was her live demo of how cheap this is — copy the block, change two lines:

```
- path: /home
  pathType: Prefix
  backend:
    service:
      name: yatri-backend-service-1
      port:
        number: 80
```

⚠ WATCH OUT

Her one hard rule when copy-pasting path blocks: *"Always make sure that all the services that you are specifying in your YAML file must be unique."* Two path blocks pointing at the same service name is almost always a copy-paste you forgot to finish.

🧩 WHY IT WORKS

Why `rewrite-target: /$2` exists — the annotation she showed but didn't unpack.

Without it, a request for `yatri.local/api/orders` reaches your backend pod **still spelled** `/api/orders`. But your Python app only defines a route called `/orders` — it never heard of `/api`. Result: 404, and a confusing one, because the Ingress *did* route correctly.

```
browser:  /api/orders
          |  |
          |  |
          $1 $2      ← the two regex capture groups in /api(/|$)(.*)

rewrite-target: /$2 → backend receives: /orders  ✓
no rewrite-target → backend receives: /api/orders  ✗ 404
```

That's also why `use-regex: "true"` and `pathType: ImplementationSpecific` are there — capture groups only work with regex paths enabled.

8.2 Host-based routing

Same idea, but now the **subdomain** changes and the path stays put.

```
portal.campus.local ┌───┐
                    │   │
                    │   └── ingress ──┐
                    │                   │
api.campus.local   └───┘                 └── yatri-backend-service
```

🗣️ SAID IN CLASS

Her explanation of subdomains, via her own site: `campus.local` is the main domain; `portal` and `api` are **subdomains**. "Our website `yatricloud.com` is the domain. And I have my portfolio deployed on `nancy.yatricloud.com` — that is my subdomain. I can create multiple subdomains. When you are using the subdomain inside a host, that becomes host-based routing."

From her `03-ingress/ingress-tls.yaml`:

```
spec:
  ingressClassName: nginx
  rules:
    # Host 1: the student portal
    - host: portal.campus.local
      http:
        paths:
          - path: /()(.* )
            pathType: ImplementationSpecific
            backend:
              service:
                name: yatri-frontend-service
                port:
                  number: 80

    # Host 2: the REST API
    - host: api.campus.local
      http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: yatri-backend-service
                port:
                  number: 80
```

NOTE

One transcript garble corrected here. On the audio she says `api.campus.local` redirects to "yatri frontend service one" — the ASR mangled it and she was reading fast. **Her actual file sends it to `yatri-backend-service`**, which is also the only thing that makes sense. The YAML above is from her repo, so it's the version to trust.

| 8.3 Hybrid — and the comparison

Asked "how can we implement both?", the class got there: put **multiple paths under a host**, then add a second host.

```
rules:
- host: portal.campus.local      # host-based ↵
  http:
    paths:
      - path: /                  # path-based ↵ both at once
        ...frontend
      - path: /api               # path-based ↵
        ...backend
- host: api.campus.local        # host-based
...
```

	Path-based	Host-based	Hybrid
What changes	the path	the subdomain	both
Example A	yatri.local/	portal.campus.local	portal.campus.local/
Example B	yatri.local/api	api.campus.local	portal.campus.local/api
Needs DNS per route?	❌ one hostname	✅ one per subdomain	✅
Typical use	one app, many routes	separate apps/teams	large products

💡 GOOD TO KNOW

TLS lives in the host-based file for a reason. Her `ingress-tls.yaml` adds a `tls:` block naming both hosts and a `secretName: campus-tls-cert`. That secret is a `kubernetes.io/tls Secret` — the typed variant mentioned in §4.3 — created from a cert and key:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=campus.local/O=CampusDevOps"

kubectl create secret tls campus-tls-cert --cert=tls.crt --key=tls.key
```

Then `kubectl get ingress` shows `PORTS 80, 443` — confirmation that HTTPS is live. **This ties §4 and §8 together:** the Ingress is where one certificate protects every service behind it, and it's stored as a Secret.



09 🚦 Ingress vs Ingress Controller

This is the homework she set — and then, being unable to resist, mostly answered.

SAID IN CLASS

"When we create this `ingress.yaml` file — have you checked, we only implemented rules. Path-based routing and host-based routing inside this `rules` field. Can you see this `rules` field? So basically in `ingress.yaml` we only specify the rules. That's why we are creating `ingress.yaml` — to specify the rules. Who is going to implement those rules?"

Answer from the room: **Ingress controller**. Correct.

`ingress.yaml`

```
"/ → frontend"
"/api → backend"
```

the RULEBOOK 📖
(does nothing by itself)

—reads—>

Ingress Controller

```
an actual NGINX pod
running in your cluster
```

| actually moves
▼ the packets
frontend / backend pods

LOCK THIS IN

Homework answered — Ingress vs Ingress Controller.

	Ingress	Ingress Controller
What it is	a Kubernetes object (YAML you write)	a pod running a reverse proxy
What it does	declares routing rules	enforces them — actually proxies traffic
Ships with Kubernetes?	✅ the API type does	❌ you install it yourself
Analogy	the traffic-sign plan on paper	the traffic police officer at the junction
If missing	no routes defined	your rules do nothing at all

⚠️ **The consequence that catches everyone:** an `Ingress` with no controller installed will `apply` successfully, show up in `kubectl get ingress`, and **route nothing**. No error. It just sits there with an empty `ADDRESS` column.

NGINX Ingress Controller is the most common implementation (others: Traefik, HAProxy, AWS ALB Controller). On Minikube it's one command:

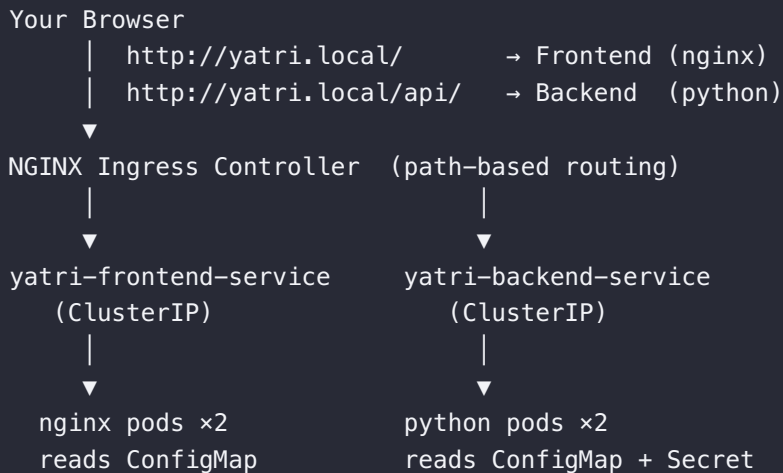
```
minikube addons enable ingress
kubectl get pods -n ingress-nginx # confirm the controller pod is Running
```

On EKS you'd install `ingress-nginx` via Helm, and AWS provisions one NLB for it.

**10**

Lab 2 — The Full Demo, All Three Together

Her `04-full-demo` folder: two services, a ConfigMap, a Secret and an Ingress, wired up.



10.1 The `---` trick she taught in passing

Opening `backend.yaml`, she pointed out something students hadn't seen: a **Deployment and its Service in one file**.



SAID IN CLASS

"You don't know how to add the `service.yaml` inside your `deployment.yaml`. This is the way — `---`. So when the Kubernetes API server reads your YAML file and validates it, it sees `---` means you have completed the file that it's reading, and a new file will be there. The same content I'm specifying inside my `service.yaml`, I just copy-paste over here — but before that you need to mention `---`."

```
apiVersion: apps/v1
kind: Deployment
# ... the whole deployment ...
---                                # ← YAML document separator
apiVersion: v1
kind: Service
# ... the whole service ...
```

💡 GOOD TO KNOW

It's three hyphens, not one. The ASR heard "hyphen" (singular) throughout; `---` is the YAML document separator and one `-` won't work. One `kubectl apply -f backend.yaml` then creates **both** objects, and the output confirms it:

```
deployment.apps/yatri-backend created
service/yatri-backend-service created
```

10.2 How the values actually get into the pod

This is the payoff of the whole class, and it's two different mechanisms in one file:

```
# ALL FIVE ConfigMap keys at once
envFrom:
  - configMapRef:
      name: yatri-app-config

# Secret keys, ONE BY ONE
env:
  - name: POSTGRES_USER
    valueFrom:
      secretKeyRef:
        name: yatri-db-secret
        key: POSTGRES_USER
  - name: POSTGRES_PASSWORD
    valueFrom:
      secretKeyRef:
        name: yatri-db-secret
        key: POSTGRES_PASSWORD
```

✅ LOCK THIS IN

The two injection styles — worth knowing both by name.

	<code>envFrom</code> + <code>configMapRef</code>	<code>env</code> + <code>secretKeyRef</code>
Pulls	every key in the object	one named key
Verbosity	3 lines total	6 lines per variable
Good for	bulk config	picking exactly what a container is allowed to see

Both end up as ordinary Linux environment variables inside the container — which is why `env | grep` in the next step proves it.

DO THIS YOURSELF

Do this yourself — the full manual run. She explicitly told the class **not** to run `run-demo.sh` first: "I want to see one error in your laptop. If there's not an error, then I'll understand that you've run the SH file." Do it by hand first.

Step 1 — the controller.

```
minikube addons enable ingress
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=120s
```

Step 2–5 — the five applies, in her order:

```
kubectl apply -f frontend.yaml
kubectl apply -f backend.yaml
kubectl apply -f configmap.yaml
kubectl apply -f secret.yaml
kubectl apply -f ingress.yaml
```

Step 6 — confirm everything landed.

```
kubectl get configmap yatri-app-config
kubectl get secret yatri-db-secret
kubectl get pods -l app=yatri-frontend
kubectl get pods -l app=yatri-backend
kubectl get svc yatri-frontend-service yatri-backend-service
kubectl get ingress yatri-ingress
```

Wait until `ADDRESS` is populated on the Ingress:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
yatri-ingress	nginx	yatri.local	192.168.49.2	80	15s

Step 7 — teach your laptop what `yatri.local` means. This is the step everyone missed:

```
sudo nano /etc/hosts
```

Add one line — Minikube's IP, then the hostname from your `ingress.yaml` :

```
192.168.49.2 yatri.local
```

Or in one command:

```
echo "$(minikube ip) yatri.local" | sudo tee -a /etc/hosts
```

Step 8 — test both routes.

```
curl http://yatri.local/  
curl http://yatri.local/api/
```

The `/api/` response is the one that proves everything:

```
Yatri Backend API  
=====
```

ENVIRONMENT	: production	← from the ConfigMap
LOG_LEVEL	: INFO	← from the ConfigMap
DEFAULT_CURRENCY	: INR	← from the ConfigMap
POSTGRES_USER	: yatri_admin	← from the Secret
POSTGRES_DB	: yatri_production_db	← from the Secret

Step 9 — verify from inside the pod.

```
kubectl exec -it deploy/yatri-backend -- env | grep -E 'ENVIRONMENT|LOG_LEVEL|  
POSTGRES'
```

🧩 WHY IT WORKS

Why `/etc/hosts` is needed at all. `yatri.local` is not a real domain — no DNS server on Earth knows it. `/etc/hosts` is your machine's private phonebook, checked **before** DNS. Adding that line is how your browser learns " `yatri.local` means 192.168.49.2". Without it, `curl http://yatri.local` fails at name resolution and never even reaches the Ingress — which looks exactly like a broken Ingress but isn't.

💡 **The other way**, no `sudo` needed — send the hostname as a header instead:

```
curl -s -H "Host: yatri.local" http://$(minikube ip)/api/
```

CLARIFIED

"It won't work on your laptop, only on Ubuntu" — clarified.

🗣️ She told the class the demo *"maybe in Ubuntu it will work, other laptop it will not work"*, and asked *"how many of you are running this command in server?"*

It isn't really about Ubuntu vs. other OSes. On macOS (and Windows), Minikube/Docker Desktop runs the cluster inside a **hidden Linux VM**. `minikube ip` returns an address that exists *inside that VM* and is not routable from your host, so `/etc/hosts` pointing at it can't help. On native Linux, the cluster shares your machine's network, so it just works.

The fix on a Mac is not "use Ubuntu" — it's to open a tunnel:

```
minikube tunnel          # in a second terminal, needs sudo; then yatri.local works
```

📌 This is the **same root cause** as the `--network host` issue in Class 8 §10 and the `minikube ip` issue in Class 13 §5 — three symptoms, one Linux VM. **In her assessment, answer as she teaches it: run the demo on the lab server.**

10.3 The automation script

Once you've done it by hand, her `run-demo.sh` does all nine steps — including waiting for the controller and patching `/etc/hosts` :

```
chmod +x run-demo.sh
./run-demo.sh
```

SAID IN CLASS

"It's an automation script that I created. If you run this, you don't need to do the `kubectl apply` of any command. It will automatically apply all the commands, add the ingress, add the hostname in `/etc/hosts` , and your complete service will be up and running — even it will give you the output that you need to see this URL."

And when you're done:

```
./cleanup.sh          # deletes ingress → backend → frontend → secret → configmap
```

⚠ WATCH OUT

She said "don't run cleanup" during class — for the obvious reason that it deletes the thing you just built. Run it at the *end*, then verify:

```
kubectl get configmap yatri-app-config # Error from server (NotFound)
kubectl get ingress yatri-ingress     # Error from server (NotFound)
```

📌 NOTE

A small inaccuracy in her own repo, worth knowing before it confuses you.

`04-full-demo/README.md` says the **frontend** "reads `LOG_LEVEL` and `ENVIRONMENT` from a `ConfigMap`". The `ConfigMap` **is** injected into the frontend pod (`envFrom` in `frontend.yaml`), so `kubectl exec` into it and you'll see the variables. But the image is stock `nginx:1.25-alpine`, which **never reads them** — so the page at `/` is just the default "Welcome to nginx!" and shows no config values. Only the **backend** actually prints them. If `/` doesn't display your `ConfigMap`, nothing is broken.

Two smaller slips in the same folder: that `README` lists `POSTGRES_DB: 22 bytes` (it's **19** — `yatri_production_db` is 19 characters, and her `lab.md` has it right), and `lab.md` Part 8 says the wrongly-encoded `secretpassword` "ends in `Ao=`" — that's carried over from the `mypassword` example; the real output is `c2VjcmlV0cGFzc3dvcmQK`, ending in **K**. The lesson in §11 is unaffected.



11 ⚠ The Trailing-Newline Bug

📌 NOTE

Sourcing note. This didn't come up in the live audio — it's from her `troubleshooting/secret-base64-gotcha.md` and `lab.md` Part 8, which are part of this session's material. It's the single most common `Secret` bug and it's five minutes well spent.

The symptom: Postgres rejects your app with `FATAL: password authentication failed for user "yatri_admin"` — and the developer swears the password is right.

The cause: `echo` adds an invisible newline.

```
echo "mypassword" | xxd
```

```
00000000: 6d79 7061 7373 776f 7264 0a      mypassword.
                ▲▲
                that's \n – a real character
```

So the app receives `mypassword\n` (11 bytes) instead of `mypassword` (10 bytes), and the database is correct to refuse it.

The fix — one flag:

```
echo "secretpassword" | base64      # ✗ c2Vjc mV0cGFzc3dvcmQK
echo -n "secretpassword" | base64   # ✓ c2Vjc mV0cGFzc3dvcmQ=
                                   # ▲ the -n suppresses the newline
```

✓ LOCK THIS IN

Always `echo -n` when encoding a secret. No exceptions. The failure is silent, the error message points at the wrong thing, and you will lose an afternoon to it.

✍ TRY IT YOURSELF

Try it yourself — prove the invisible character exists.

```
echo "c2Vjc mV0cGFzc3dvcmQK" | base64 --decode
```

Watch your cursor **jump to the next line** after the output. That jump *is* the `\n`. Now the correct one:

```
echo -n "secretpassword" | base64 | base64 --decode; echo "|"
```

The `|` prints on the same line — no stray newline.



12 Updating a ConfigMap — The Pods Don't Care

NOTE

Also from her [lab.md \(Part 9\)](#), not the live audio — but it's the direct consequence of the warning in §3.4 and it's a classic interview follow-up.

Change a value on a running ConfigMap:

```
kubectl patch configmap yatri-app-config --type merge \
  -p '{"data":{"ENVIRONMENT":"staging"}}'
```

Now ask the running pod what it thinks:

```
kubectl exec -it deployment/yatri-backend -- env | grep ENVIRONMENT
```

```
ENVIRONMENT=production      ← unchanged!
```

WHY IT WORKS

Why. Environment variables are handed to a process **once, at container start**. The ConfigMap in etcd changed; the already-running process's environment did not. Nothing will ever push the new value into a live process.

The fix — restart the pods:

```
kubectl rollout restart deployment/yatri-backend
kubectl rollout status deployment/yatri-backend
kubectl exec -it deployment/yatri-backend -- env | grep ENVIRONMENT
```

```
ENVIRONMENT=staging        ← now it's live
```

INTERVIEW ALERT

Interview alert. "You updated a ConfigMap and the app still shows the old value. Why?" — because env vars are set at container start; you need a rolling restart. **Bonus point:** if the ConfigMap is **mounted as a volume** instead of injected as env vars, the file on disk *does* update automatically (after a sync delay) — though your app still has to re-read the file.



13



Cheat-Sheet

ConfigMap & Secret

Command	What it does
<code>kubectl apply -f configmap.yaml</code>	Create/update a ConfigMap
<code>kubectl apply -f secret.yaml</code>	Create/update a Secret
<code>kubectl get configmap</code> / <code>kubectl get cm</code>	List ConfigMaps (<code>DATA</code> = number of keys)
<code>kubectl get secrets</code>	List Secrets (<code>TYPE</code> = <code>Opaque</code> , <code>DATA</code> = number of keys)
<code>kubectl get configmap,secrets</code>	Both at once — no space after the comma
<code>kubectl describe configmap <name></code>	Show all keys and values
<code>kubectl describe secret <name></code>	Show keys and byte counts only — values masked
<code>kubectl get cm <name> -o jsonpath='{.data.KEY}'</code>	Read one specific key
<code>kubectl get secret <n> -o jsonpath='{.data.K}' \ base64 --decode</code>	Decode one secret value
<code>kubectl patch configmap <n> --type merge -p '{"data":{...}}'</code>	Change a value in place
<code>kubectl rollout restart deployment/<name></code>	Make pods pick up the new values
<code>kubectl delete configmap <name> / delete secret <name></code>	Remove it

base64

Command	Result
<code>echo -n "text" \ base64</code>	✅ encode, no trailing newline

Command	Result
<code>echo "text" \ base64</code>	✗ encodes a hidden <code>\n</code> too
<code>echo -n "<b64>" \ base64 --decode</code>	Decode back to plain text
<code>echo "text" \ xxd</code>	See the raw bytes, including the <code>0a</code>

Ingress

Command	What it does
<code>minikube addons enable ingress</code>	Install the NGINX Ingress Controller
<code>kubectl get pods -n ingress-nginx</code>	Confirm the controller pod is <code>Running</code>
<code>kubectl apply -f ingress.yaml</code>	Apply the routing rules
<code>kubectl get ingress</code>	List — wait for <code>ADDRESS</code> to populate
<code>kubectl describe ingress <name></code>	See every host → path → backend mapping
<code>kubectl create secret tls <n> --cert=tls.crt --key=tls.key</code>	TLS cert for HTTPS termination
<code>minikube tunnel</code>	Make the Ingress reachable on macOS/Windows
<code>curl -H "Host: yatri.local" http://\$(minikube ip)/</code>	Test without touching <code>/etc/hosts</code>

YAML field reference

Field	Object	Note
<code>data:</code>	ConfigMap, Secret	replaces <code>spec:</code>
<code>type: Opaque</code>	Secret	the one extra line vs a ConfigMap
<code>type: kubernetes.io/tls</code>	Secret	for TLS cert + key
<code>apiVersion: networking.k8s.io/v1</code>	Ingress	not <code>v1</code> , not <code>apps/v1</code>
<code>ingressClassName: nginx</code>	Ingress	mandatory — omit it and nothing routes
<code>rules:</code>	Ingress	host → paths → backend service
<code>pathType:</code>	Ingress	<code>Prefix</code> · <code>Exact</code> · <code>ImplementationSpecific</code>
<code>envFrom: configMapRef:</code>	Pod spec	inject all keys

Field	Object	Note
<code>env: valueFrom: secretKeyRef:</code>	Pod spec	inject one key
<code>---</code>	any	YAML document separator — two objects, one file

The recap-drill answers

Question	Answer
Master node components	4 — etcd, API server, scheduler, controller-manager
Worker node components	2 — kubelet, kube-proxy (+ containerd via CRI)
Only point of contact between components	API server
Five Service types	ClusterIP · NodePort · LoadBalancer · ExternalName · Headless
ClusterIP vs Headless, in YAML	Headless = <code>clusterIP: None</code>
Which type shows EXTERNAL-IP	ExternalName (<code>kubectl get svc -o wide</code>)
LoadBalancer locally	<code><pending></code>
Deployment vs StatefulSet	stateless vs stateful; StatefulSet gives stable names (<code>mysql-0</code>)
DaemonSet	one pod on every node
Deployment strategies	4, starting with rolling update



14



Homework

She said *"three things"* and then *"two tasks"* — here is everything actually assigned, in priority order.

- ☐ **1. Overdue: all Kubernetes tasks so far.** In your repo's `README.md`, at the link she already shared. **Was due EOD 17 Sep.**
- ☐ **2. The five Service types screenshot.** Output for **NodePort, ClusterIP, LoadBalancer, ExternalName and Headless**, plus a final `kubectl get svc`. Into `README.md`, repo link in chat.
- ☐ **3. Research: what is an Ingress Controller?** And specifically the **NGINX Ingress Controller**.
- ☐ **4. Answer: what is the difference between Ingress and Ingress Controller?** → §9 has it, but write it in your own words first.
- ☐ **5. Run the full demo yourself** — `04-full-demo`, manually first (§10), then via `run-demo.sh`. **Due tomorrow (18 Sep).**

GOOD TO KNOW

Still outstanding from earlier classes, in case you're catching up: `journalctl` (Class 2), soft vs hard links (Class 2), `adduser` vs `useradd` (Class 2), and the Class 12 rollout tasks.



15 Notes on the Transcript Itself

The instructor. The transcript header credits "**Ritesh Prajapati**". That is the developer of the *Scaler++ Chrome extension* that downloaded the file — **not** the instructor. This class, like all of them, was taught by **Nensi Ravaliya** (heard as "*Nancy*" throughout the ASR; she writes `Nensi`, and her domain `yatricloud.com` shows up in the demo hostnames).

The lecture title is wrong — third time running. Header and folder both say "**Helm**". Helm was mentioned once, in a single sentence, as somewhere you must not store secrets. The real content is ConfigMaps, Secrets and Ingress — which is what the **Class 12** folder is named. The topics keep sliding one or two folders behind their labels.

Transcription quality. Header shows `Model use: groq, whisper-large-v3-turbo`, and where she speaks English it's clean. Two stretches are badly damaged:

- **The controller-names drill (~4 minutes)** collapses into unreadable Devanagari-script noise mid-answer. The questions and the answers that were clearly audible are preserved in §2.1; the rest is genuinely unrecoverable and has not been guessed at.
- **The ~15 minutes of lab time** after the ConfigMap/Secret hands-on is mostly ambient room noise transcribed as Indonesian ("*Sampai jumpa di video selanjutnya*"), repeated "*Thank you*", and fragments of half-heard side conversations. Nothing technical was lost — she was walking between desks.

Folded in rather than dropped: her Hindi/Gujarati asides are translated inline where they carried meaning — the prerequisites warning in §1, the Akbar–Birbal narration in §7, and the closing exchange where a student asked her to re-explain and she pointed out she'd *already* explained it via Akbar and Birbal.

Skipped as pure noise: the repeated "*Thank you*" / "*foreign*" blocks, the Indonesian sign-off phrases, the Portuguese fragments during the recap drill, and the roll-call banter while she chased repo links round the room (partly kept in §1 because it's the reason the class opened the way it did).

Garbled-term corrections:

Transcript says	Actually
"hemchat" / "Hemchat folder"	Helm chart
"as your keyword" / "Azure keyword"	Azure Key Vault
"HashiCorp world" / "HashiCorp Keyword"	HashiCorp Vault

Transcript says	Actually
"kubectl will send the heartbeat"	kubelet
"Qproxy and Qblet"	kube-proxy and kubelet
"yathri" / "yatri" / "Yadri" / "y3"	yatri (her brand — yatricloud.com)
"NancyRavalia.me"	her own domain (nancy.yatricloud.com)
"hyphen" (for splitting YAML docs)	--- — three hyphens
"data is free"	data is three
"Friendler" (API server's job)	handler
"demon set"	DaemonSet
"not put" / "noteport"	NodePort
"cad command" / "call command"	curl command
"chmol plus x"	chmod +x
"SS file" / "run demo SS"	.sh file — run-demo.sh
"mini cube"	Minikube
"ADO library"	Azure DevOps Library (variable groups)
"Sahista"	her college professor's name, spelling unverified

Cross-referenced against her repo. Every YAML block, expected output and command in §3.3, §4.3, §6, §8, §10, §11 and §12 was checked against [session-12-ingress-configmaps-secrets](#) rather than transcribed from audio. The base64 values and byte counts in §4.3 and §6 were independently decoded and verified. Three small errors in her own repo files are flagged in §10.3 — the frontend/ConfigMap claim, a **22 bytes** that should be **19** , and an **Ao=** that should be **K** .

